

# ANPR-VISION: reconocimiento automático de matrículas para parqueaderos

Karol Natalia Osorio Poveda  
Neiva, Huila, Colombia  
nataliaosorio973@gmail.com

**Resumen**—La expansión de la cantidad de vehículos en las ciudades latinoamericanas ha aumentado la demanda de espacios de parqueo, especialmente en instituciones educativas, centros comerciales y conjuntos residenciales. En muchos de estos contextos, la operación del parqueadero sigue dependiendo de procesos manuales, como el registro en papel o en hojas de cálculo, y de dispositivos de control de acceso poco integrados con el resto del sistema de información. Esto dificulta saber, casi en tiempo real, cuántos vehículos hay realmente parqueados, genera filas en los puntos de ingreso y salida y limita la posibilidad de realizar análisis históricos sobre patrones de uso y comportamiento del recaudo.

Los sistemas de Reconocimiento Automático de Matrículas (*Automatic Number Plate Recognition*, ANPR) permiten automatizar el registro de vehículos a partir de video, sin depender de que una persona tome nota de cada placa. Hoy en día, gracias a los avances en inteligencia artificial y visión por computador, ya es posible detectar y reconocer matrículas con bastante precisión, incluso cuando la luz no es la mejor o la cámara no está en el ángulo ideal. Aun así, muchas de las herramientas comerciales siguen siendo de código cerrado, tienen licencias costosas y casi no explican cómo están hechas, lo que complica usarlas para experimentar, aprender o ajustarlas a nuevas necesidades.

En este trabajo se presenta el diseño de una arquitectura de sistema ANPR orientada a la gestión de parqueaderos, construida íntegramente con tecnologías de código abierto. La propuesta se centra en un microservicio de visión, implementado en Python, que utiliza modelos de la familia *YOLO* para detectar vehículos y matrículas, bibliotecas de OCR para el reconocimiento de caracteres (por ejemplo, EasyOCR) y algoritmos de seguimiento como ByteTrack para mantener la identidad del vehículo a lo largo del tiempo. Los eventos generados por el microservicio se publican en un bus de mensajería distribuido basado en Apache Kafka y son consumidos por un sistema de gestión de parqueaderos desarrollado en .NET, que expone una API para registrar entradas, salidas y transacciones. Sobre esta base, las interfaces web y móviles, construidas con tecnologías modernas de frontend, reciben notificaciones en tiempo casi real mediante canales bidireccionales como SignalR.

El desarrollo forma parte de un trabajo de grado del Servicio Nacional de Aprendizaje (SENA) y tiene un enfoque principalmente educativo y de experimentación. El sistema, llamado ANPR-VISION, se implementa como un prototipo funcional y se prueba en un escenario controlado, lo que permite revisar con cuidado hasta dónde llega la arquitectura propuesta y cuáles son sus

principales limitaciones. En el artículo se presentan de forma clara los componentes principales del sistema y cómo se comunican entre sí. También se hablan de ideas para seguir mejorando la solución, por ejemplo agregar módulos que ayuden a entender mejor cómo se usa el parqueadero o permitir que el sistema administre varios parqueaderos al mismo tiempo. La intención es que ANPR-VISION pueda servir como base para otros proyectos de parqueaderos inteligentes y control de acceso de vehículos, sobre todo en espacios académicos y en escenarios donde el presupuesto es limitado.

**Index Terms**—reconocimiento automático de matrículas, ANPR, gestión de parqueaderos, microservicios, DevOps

## I. INTRODUCCIÓN

En muchas ciudades, moverse en carro se ha vuelto cada vez más difícil. Cada año hay más vehículos, pero las vías y la infraestructura siguen casi iguales, y al mismo tiempo se intenta que el transporte genere menos contaminación y menos impacto en la ciudad. Una parte importante de esa congestión no se da solo mientras se conduce, sino también cuando las personas buscan dónde parquear y se encuentran con una gestión poco eficiente de los parqueaderos. En muchos casos todavía se utilizan procesos manuales o sistemas que no están conectados entre sí. Esto termina generando filas en las entradas, respuestas tardías sobre si hay cupos disponibles y muy poco uso de los datos históricos para entender cómo se están usando realmente estos espacios.

En este contexto, los sistemas de Reconocimiento Automático de Matrículas (*Automatic Number Plate Recognition*, ANPR) se usan cada vez más para automatizar la entrada y salida de vehículos. En pocas palabras, una cámara lee la placa y el sistema registra el ingreso o la salida sin que la persona tenga que hacer un trámite manual. Cuando estos sistemas se integran con un parqueadero “inteligente”, es posible asociar las placas con usuarios o reservas y consultar con más detalle cómo se están usando los cupos a lo largo del tiempo. Aun así, muchas de las soluciones ANPR que se ofrecen en el mercado siguen siendo de código cerrado, requieren licencias costosas y dejan al cliente muy dependiente de un proveedor específico. Además, no suelen adaptarse bien cuando se quieren usar en universidades, en proyectos de investigación o en prototipos que necesitan cambiar y mejorar con rapidez.

Al mismo tiempo, muchas organizaciones han empezado a trabajar con prácticas de *DevOps* y con esquemas de integración y entrega continua (CI/CD), que han cambiado la forma de construir y publicar aplicaciones de software, especialmente en arquitecturas basadas en microservicios. La idea principal es automatizar lo más posible las pruebas, la compilación y el despliegue, de manera que los cambios en el código lleguen más rápido y con menos errores a producción **devops\_tools\_2019, ci\_cd\_quality\_2025**. Sin embargo, cuando el sistema tiene que trabajar con imágenes y con modelos de inteligencia artificial más complejos, aplicar este tipo de automatización ya no es tan sencillo. En la práctica, estos modelos suelen ser más pesados, tardan más en ejecutarse y necesitan computadores más potentes (por ejemplo, con tarjeta gráfica dedicada), lo que hace mucho más difícil diseñar y mantener un proceso automático de pruebas y despliegue que sea estable.

En este trabajo se propone una arquitectura completa para un sistema ANPR enfocado en la gestión de parqueaderos, basada en tecnologías de código abierto y apoyada en un flujo de trabajo DevOps que facilita su despliegue y su evolución en el tiempo. El sistema se toma como caso de estudio bajo el nombre **ANPR-VISION**, pero la descripción se plantea de forma lo bastante general como para que pueda servir de guía en otros proyectos de parqueadero inteligente o de control de acceso vehicular.

#### *I-A. Problema de investigación*

A partir de lo que se ha encontrado en trabajos previos y de lo que se ve en el funcionamiento real de muchos parqueaderos, surge la siguiente pregunta central:

¿Cómo diseñar una arquitectura para un sistema ANPR, basada en tecnologías abiertas, que permita automatizar la gestión de parqueaderos y que, al mismo tiempo, pueda desplegarse y mantenerse de forma ágil usando prácticas de DevOps?

Esta pregunta se conecta, en la práctica, con dos tipos de retos. El primero tiene que ver con la parte técnica de detectar y leer matrículas en condiciones reales, donde la luz cambia, la cámara no siempre está bien ubicada o los vehículos se mueven rápido. El segundo está relacionado con la operación del sistema: debe estar disponible todo el tiempo y poder crecer cuando aumenta la demanda, procesando video y eventos en tiempo real sin que el servicio se detenga.

#### *I-B. Objetivo general*

Diseñar y explicar una arquitectura de referencia para un sistema de Reconocimiento Automático de Matrículas orientado a la gestión inteligente de parqueaderos, usando herramientas de código abierto y prácticas de DevOps que hagan más sencillo su despliegue, actualización y mantenimiento.

#### *I-C. Objetivos específicos*

- Reunir y comentar los trabajos más relevantes sobre reconocimiento automático de matrículas, parqueaderos inteligentes y control de acceso de vehículos.
- Explicar de manera clara qué partes debería tener un sistema ANPR para parqueaderos y cómo se comunican entre sí.
- Plantear una forma ordenada de construir, probar y poner en funcionamiento el sistema, apoyándose en herramientas de automatización.
- Desarrollar un prototipo que funcione y usarlo como ejemplo para mostrar qué tan viable es la propuesta.
- Señalar qué aporta la propuesta frente a otras soluciones, qué limitaciones tiene y qué se podría mejorar en trabajos futuros.

#### *I-D. Estructura del artículo*

El artículo se divide en varias secciones. En la Sección ?? se presentan los trabajos previos sobre ANPR, parqueaderos inteligentes y el uso de DevOps en este tipo de sistemas. La Sección ?? explica el enfoque que se siguió para plantear y concretar la arquitectura. En la Sección ?? se describe con más detalle la solución propuesta y el caso de estudio construido. La Sección ?? reúne los principales resultados y los criterios usados para evaluarlos. Finalmente, la Sección ?? recoge las conclusiones más importantes y sugiere algunas líneas posibles para trabajos futuros.

## II. TRABAJOS RELACIONADOS

Hay muchos trabajos publicados sobre Reconocimiento Automático de Matrículas (ANPR) y sobre sistemas para gestionar parqueaderos de forma más automática. Algunos trabajos se centran en cómo integrar ANPR dentro de sistemas que ya gestionan parqueaderos, mientras que otros se enfocan en mejorar los algoritmos que detectan y reconocen las placas usando técnicas de aprendizaje profundo. En esta sección se hace un resumen de los aportes más importantes, organizados en tres grupos: (i) sistemas ANPR usados en la gestión de parqueaderos y en iniciativas de ciudades inteligentes; (ii) enfoques para la detección y el reconocimiento de matrículas; y (iii) experiencias basadas en software libre y en la adaptación a formatos de placas de distintas regiones, que son especialmente valiosas para proyectos educativos como ANPR-VISION.

#### *II-A. ANPR en sistemas de aparcamiento inteligente y ciudades inteligentes*

Una primera línea de trabajos se enfoca en usar ANPR dentro de soluciones de aparcamiento inteligente (*smart parking*). En **streetsavvy\_2024**, por ejemplo, se presenta *StreetSavvy*, un sistema que automatiza la gestión del parqueadero usando reconocimiento de placas y tarifas que van cambiando según el tiempo de uso y las condiciones del momento. La idea es sencilla: el sistema usa cámaras para identificar los vehículos que entran al parqueadero,

calcula la tarifa según el tiempo que permanecen y otras condiciones del contexto, y le muestra a los administradores un panorama detallado de cómo se están usando las plazas. Su aporte principal es enseñar cómo el reconocimiento de matrículas se puede conectar con modelos de cobro más avanzados, aunque la descripción técnica de la arquitectura se mantiene en un nivel general.

En el ámbito académico, **campussense\_2017** presenta *CampusSense*, un sistema pensado para monitorear y gestionar parqueaderos en campus universitarios. Allí se usa ANPR para registrar la ocupación y mejorar la experiencia de las personas que buscan dónde parquear. La solución combina cámaras, servidores de procesamiento y una interfaz de administración que muestra, casi en tiempo real, cuántos cupos hay disponibles. Aunque el estudio se centra en un campus concreto, deja en evidencia que ANPR puede ayudar a reducir controles manuales y a planear mejor la capacidad de estacionamiento.

Otros trabajos combinan ANPR con tecnologías de Internet de las Cosas (IoT), es decir, con redes de sensores y dispositivos conectados. En **iot\_parking\_ocr\_2023** se propone un sistema para monitorear parqueaderos que usa sensores y cámaras, y aplica OCR para leer las placas. De forma similar, **smart\_parking\_wrongly\_parked\_2023** presenta un sistema de parqueadero inteligente que detecta vehículos mal estacionados y sugiere cupos disponibles, aprovechando la información de cámaras y sensores para hacer cumplir mejor las normas y mejorar la experiencia de los usuarios.

En trabajos sobre ciudades inteligentes, **realtime\_parking\_two\_cameras\_2024** muestra un sistema de parqueadero en tiempo real que pone dos cámaras en cada entrada. La idea es que, si un carro queda tapado en una cámara, la otra lo pueda ver y así el sistema falle menos al detectar los vehículos. Por otro lado, **smart\_parking\_lpr\_efficiency\_2024** estudia cómo leer las placas en tiempo real puede ayudar a organizar mejor los parqueaderos y el tráfico de la ciudad, y cómo este tipo de sistemas se puede conectar con otras plataformas que ya manejan distintos servicios urbanos.

En conjunto, estos trabajos muestran que, al combinar ANPR con sistemas de gestión de parqueaderos, es posible automatizar tareas que antes se hacían de forma manual, como registrar entradas y salidas, asignar plazas o calcular tarifas. Sin embargo, en muchos casos la descripción de la solución se queda en diagramas de alto nivel: no siempre se explican con detalle los componentes, los mecanismos de comunicación entre ellos o las decisiones de diseño necesarias para que otros puedan replicar la arquitectura en contextos distintos.

## II-B. Enfoques de detección y reconocimiento de matrículas

Una segunda línea de trabajos se concentra en los métodos que se usan para detectar y reconocer las matrículas

en las imágenes. Con el avance del aprendizaje profundo, muchas propuestas han dejado atrás las técnicas clásicas de procesamiento de imágenes y han pasado a arquitecturas que combinan detectores de objetos con modelos de OCR entrenados para distintos tipos y formatos de placa.

En **realtime\_ocr\_wsn\_its\_2019** se presenta un sistema de reconocimiento de placas en tiempo real para sistemas de transporte inteligente (ITS). Allí se combina OCR con redes de sensores inalámbricos, buscando tiempos de respuesta lo suficientemente bajos como para usar la solución en carretera. Para lograrlo, se apoya en hardware y redes de comunicación que permiten repartir la carga de procesamiento. De manera complementaria, **anpr\_pakistan\_2020** propone un método eficiente para el reconocimiento de matrículas en Pakistán, ajustando los modelos y sus parámetros a la apariencia específica de las placas de ese país, lo que refuerza la idea de que estos sistemas deben adaptarse a cada región.

Los detectores basados en *YOLO* han ganado un papel destacado en trabajos recientes. En **yolo\_high\_precision\_lpr\_2024** se estudia cómo combinar YOLO con técnicas de posprocesamiento específicas para mejorar la precisión en la detección de placas y reducir falsos positivos en escenarios complejos. Por su parte, **car\_model\_plate\_dl\_2024** plantea un sistema de detección en tiempo real que no solo identifica la matrícula, sino también el modelo del vehículo, apoyándose en arquitecturas profundas especializadas para cada una de estas tareas.

También se han propuesto flujos completos de ANPR basados en aprendizaje profundo. En **deep\_anpr\_pipeline\_access\_control\_2022** se describe una *pipeline* de ANPR para control de acceso vehicular, en la que se separan claramente las etapas de detección de la placa, normalización de la imagen y reconocimiento de caracteres, y se explican las decisiones de diseño tomadas en cada paso. De forma complementaria, **lpr\_ai\_dl\_2025** presenta una revisión de métodos de reconocimiento de matrículas basados en inteligencia artificial y aprendizaje profundo, destacando su mayor estabilidad frente a cambios de iluminación, ángulos de cámara o ruido, en comparación con enfoques más tradicionales.

Además, existen trabajos orientados a la divulgación y la enseñanza. El trabajo de **cv\_system\_lpr\_tutorial**, por ejemplo, explica paso a paso cómo usar visión por computador para construir un sistema de reconocimiento de matrículas, integrando herramientas de código abierto y conjuntos de datos fácilmente accesibles. Aunque este tipo de aportes no siempre introduce nuevas arquitecturas, resulta muy útil como guía para prototipos y proyectos educativos.

En general, los estudios centrados en algoritmos de detección y reconocimiento suelen presentar resultados detallados de precisión y tiempos de procesamiento, pero dedican menos espacio a explicar cómo integrar estos

modelos en sistemas completos que incluyan gestión de usuarios, almacenamiento de datos, mensajería y visualización. ANPR-VISION retoma varias de estas ideas —en particular el uso de detectores tipo *YOLO*, bibliotecas de OCR de código abierto y algoritmos de seguimiento como ByteTrack— y las inserta dentro de una arquitectura de microservicios conectados a través de un bus de mensajería, con el objetivo de aportar esa mirada de sistema completo que a menudo falta en la literatura.

### *II-C. Software libre, adaptaciones regionales y contexto educativo*

En este trabajo tiene un papel central el uso de herramientas de software libre y la posibilidad de adaptar los sistemas ANPR a los distintos formatos de placas que existen en cada región. En **software\_libre\_mercosur\_2016**, por ejemplo, se desarrolla un sistema de reconocimiento para las nuevas matrículas del Mercosur utilizando únicamente software libre. Esto permite ajustar el sistema cuando cambian los diseños de las placas y evita depender de soluciones cerradas de un solo proveedor. El estudio muestra que es posible alcanzar un desempeño adecuado con tecnologías abiertas, siempre que se tenga en cuenta la diversidad de formatos y se disponga de datos de entrenamiento suficientes y representativos.

Otros trabajos conectan el reconocimiento de matrículas con objetivos de seguridad. En **anpr\_security\_bangladesh\_2025** se propone un sistema de ANPR para mejorar la seguridad en Bangladesh, integrando la identificación de vehículos con bases de datos de listas negras y vehículos reportados. De forma complementaria, algunos sistemas de *smart parking* emplean ANPR para detectar vehículos mal estacionados o para verificar si se cumplen las normas de uso de los cupos **smart\_parking\_wrongly\_parked\_2023**. Estos enfoques muestran que ANPR no solo sirve para automatizar el cobro o el control de acceso, sino también para apoyar la supervisión y la aplicación de políticas de movilidad.

Desde el punto de vista educativo, llama la atención que haya pocos trabajos que describan, con suficiente detalle, arquitecturas ANPR pensadas explícitamente como plataforma de aprendizaje para estudiantes de programas técnicos o tecnológicos. Aunque algunas publicaciones comparten código abierto o repositorios de ejemplo, no siempre explican cómo está organizado el sistema por módulos, cómo se comunican los servicios entre sí o qué decisiones de diseño se tomaron para facilitar su comprensión por parte de equipos en formación.

En este contexto, ANPR-VISION se plantea como un caso de estudio que se apoya en estas experiencias previas, pero con un énfasis particular en la claridad de la arquitectura y en el uso de herramientas de código abierto de fácil acceso. El sistema combina un microservicio de visión implementado en Python, un backend de gestión de parqueaderos desarrollado en .NET y un frontend web

y móvil construido con tecnologías modernas, todo conectado mediante Apache Kafka y mecanismos de notificación casi en tiempo real. Aunque el proyecto se desarrolla como trabajo de grado del SENA y su alcance es acotado, la documentación de su diseño busca ofrecer una referencia que pueda ser replicada y adaptada en distintos contextos académicos y de investigación aplicada.

Finalmente, es importante señalar que varios elementos identificados en la literatura aún no se han incorporado por completo en ANPR-VISION, pero se reconocen como oportunidades de mejora. Entre ellos están la evaluación sistemática del sistema frente a conjuntos de datos públicos de diferentes regiones, la integración de módulos avanzados de analítica de ocupación y movilidad, y la exploración de despliegues en dispositivos de borde para reducir la latencia y la carga sobre el servidor central. Estas posibles extensiones se presentan en secciones posteriores como líneas de trabajo futuro coherentes con el contexto educativo del proyecto, más que como requisitos inmediatos del prototipo actual.

## III. METODOLOGÍA

El trabajo se enmarca en una investigación de tipo aplicada, orientada al diseño y construcción de una arquitectura de sistema de Reconocimiento Automático de Matrículas (ANPR) para la gestión de parqueaderos. El énfasis no está únicamente en la evaluación de un algoritmo de visión por computador, sino en la definición y validación de un conjunto de componentes que, integrados, permiten automatizar el registro de vehículos y servir como plataforma de experimentación en contextos académicos.

La investigación se desarrolló en el marco de un trabajo de grado del Servicio Nacional de Aprendizaje (SENA), dentro de un programa de formación en análisis y desarrollo de software. Por tanto, la metodología combina elementos de ingeniería de software (levantamiento de requisitos, diseño arquitectónico, implementación y pruebas) con una intención formativa: consolidar y poner en práctica conceptos de visión por computador, microservicios, mensajería distribuida y desarrollo web y móvil.

La metodología se estructura en cuatro fases principales:

1. Análisis del contexto y definición de requisitos.
2. Diseño de la arquitectura propuesta.
3. Implementación del caso de estudio ANPR-VISION.
4. Evaluación inicial y reflexión sobre resultados y extensiones posibles.

A continuación se describen en detalle cada una de estas fases, así como las tecnologías y conceptos investigados y aplicados en el proceso.

### *III-A. Fase 1: Análisis del contexto y definición de requisitos*

En la primera fase se realizó un análisis del contexto de uso al que se orienta el sistema: parqueaderos de pequeña y mediana escala, típicos de instituciones educativas, conjuntos residenciales y organizaciones que gestionan el

acceso de vehículos de manera manual o con sistemas parcialmente automatizados. Este análisis se apoyó tanto en la observación de prácticas reales como en la revisión de trabajos relacionados en ANPR, aparcamiento inteligente y ciudades inteligentes (Sección ??).

A partir de esta revisión se definió un conjunto de requisitos funcionales y no funcionales:

- **Registro automático de entradas y salidas:** el sistema debe ser capaz de detectar la entrada y salida de vehículos en puntos de acceso determinados, asociando a cada evento la matrícula reconocida, la hora y la cámara de origen.
- **Uso de tecnologías abiertas:** todos los componentes (modelos de visión, microservicios, bases de datos, frontends) deben construirse con herramientas de código abierto, de modo que el sistema pueda ser estudiado, modificado y desplegado sin licencias propietarias.
- **Arquitectura modular:** la solución debe organizarse en componentes claramente diferenciados (visión, mensajería, gestión de parqueaderos, interfaces de usuario), para facilitar su comprensión y su posible sustitución o ampliación.
- **Procesamiento en tiempo casi real:** aunque el sistema se desarrolla con fines educativos, la latencia entre la detección de un vehículo y la notificación al usuario debe ser compatible con la operación normal de un parqueadero.
- **Persistencia y consulta de información:** el sistema debe permitir almacenar los registros de visitas y ponerlos a disposición de las interfaces web y móviles para consulta posterior por parte de administradores y usuarios.
- **Escenario de validación controlado:** el prototipo debe poder desplegarse en un entorno de prueba que replique, a pequeña escala, el comportamiento de un parqueadero real, con cámaras reales o flujos de video de prueba.

Estos requisitos orientan las decisiones de diseño y acotan el alcance del proyecto: no se busca reemplazar soluciones comerciales completas, sino construir una arquitectura clara, replicable y adecuada para experimentación y formación.

### III-B. Fase 2: Diseño de la arquitectura propuesta

La segunda fase se centró en el diseño de la arquitectura de sistema. A partir de los requisitos anteriores y de la literatura consultada, se optó por una organización basada en microservicios y en mensajería distribuida, separando explícitamente la capa de visión de la lógica de gestión de parqueaderos.

En términos generales, se definieron los siguientes componentes:

1. **Microservicio de visión (Python):** responsable de recibir flujos de video desde cámaras IP o archivos de

prueba, aplicar los modelos de detección y reconocimiento de matrículas y generar eventos estructurados.

2. **Bus de mensajería (Apache Kafka):** encargado de recibir los eventos generados por el microservicio de visión y distribuirlos a los servicios consumidores; actúa como capa de desacoplamiento entre la visión y el backend de gestión.
3. **Sistema de gestión de parqueaderos (backend .NET):** API que mantiene la información de vehículos, usuarios, tarifas, plazas y registros de visitas; consume los eventos de detección para actualizar el estado del parqueadero.
4. **Interfaces de usuario (Angular y aplicación móvil):** frontends que permiten al personal operativo visualizar en tiempo cercano al real las detecciones y gestionar el parqueadero, y que ofrecen al usuario final acceso a su historial de visitas y saldos.
5. **Canal de notificación en tiempo cercano al real (SignalR):** mecanismo para enviar mensajes desde el backend hacia los clientes web y móviles cuando se produce un nuevo evento relevante (por ejemplo, detección de una matrícula o cierre de una visita).

Durante esta fase se elaboraron diagramas de componentes, de despliegue y de secuencia para representar la interacción entre los módulos. El microservicio de visión se concibió como un módulo autónomo capaz de procesar uno o varios flujos de video, mientras que el backend .NET se diseñó como la “fuente de verdad” en cuanto a la información de parqueaderos, vehículos y transacciones. Apache Kafka se ubicó en el centro de la arquitectura como canal principal de comunicación asíncrona, permitiendo que otros componentes futuros (por ejemplo, módulos de analítica avanzada) puedan suscribirse a los eventos sin modificar el microservicio de visión ni el backend existente.

En paralelo, se definieron las estructuras de datos que circulan entre los componentes, en particular el formato de los eventos de detección (matrícula, cámara, marca de tiempo, confianza, tipo de evento, etc.) y los modelos manejados en el backend (vehículos, visitas, usuarios, tarifas).

### III-C. Fase 3: Implementación del microservicio de visión

La tercera fase se centró en la construcción del microservicio de visión, implementado en Python. Esta fase incluyó el estudio y selección de los modelos y bibliotecas a utilizar, así como la integración de estos dentro de una canalización de procesamiento coherente.

Los principales elementos tecnológicos y conceptuales considerados fueron:

- **Detección de vehículos y matrículas con YOLOv5:** se adoptó un modelo de la familia *YOLOv5* entrenado para detectar vehículos y, en algunos casos, regiones de matrícula. Este tipo de modelos ofrece un equilibrio adecuado entre precisión y velocidad para aplicaciones en tiempo casi real, y cuenta con implementaciones y ejemplos de código abierto ampliamente documentados.

- **Localización de la región de la placa:** en los casos en que el modelo de detección entrega solo la caja correspondiente al vehículo, se aplica un paso adicional para localizar la matrícula dentro de dicha región, utilizando heurísticas geométricas y, en ocasiones, modelos secundarios especializados en placas.
- **Reconocimiento óptico de caracteres (OCR) con EasyOCR:** para la lectura de los caracteres de la matrícula se emplea EasyOCR, una biblioteca de OCR de código abierto que soporta múltiples idiomas y ofrece un rendimiento suficiente para el prototipo. Se investigaron aspectos como la normalización de la imagen (escala de grises, contraste, filtrado) para mejorar la calidad del reconocimiento.
- **Seguimiento de objetos con ByteTrack:** con el fin de mantener la identidad del vehículo a lo largo del tiempo, se integró el algoritmo de seguimiento ByteTrack, que asocia detecciones consecutivas basándose en la posición y en la confianza de los cuadros delimitadores. Este seguimiento permite reducir lecturas duplicadas y estimar de manera más fiable el momento de entrada y de salida.
- **Gestión de flujos de video:** el microservicio se diseñó para recibir video a través de protocolos como RTSP o desde archivos de prueba, dividiendo el flujo en fotogramas (*frames*) que se van procesando de manera secuencial. Se implementó una lógica para controlar la tasa de procesamiento (por ejemplo, procesar uno de cada varios fotogramas) y así equilibrar el consumo de recursos y la latencia.

Una vez procesados los fotogramas y reconocida la matrícula, el microservicio genera eventos estructurados que incluyen la información relevante para la capa de gestión. Estos eventos se envían a un tópico de Apache Kafka definido específicamente para detecciones de matrículas. La comunicación con Kafka se realiza mediante librerías de cliente de código abierto para Python, y se probaron diferentes configuraciones de producción (por ejemplo, tamaños de lote y políticas de reintento) en el entorno de pruebas.

Es importante señalar que, dado el carácter educativo del proyecto, la implementación se orientó a lograr un funcionamiento correcto y estable del microservicio en escenarios controlados, más que a exprimir al máximo el rendimiento del modelo o a realizar optimizaciones de bajo nivel. No obstante, la arquitectura permite sustituir el modelo de detección o la biblioteca de OCR por alternativas más avanzadas en trabajos futuros.

### III-D. Fase 4: Implementación del backend de gestión y de las interfaces de usuario

En paralelo a la construcción del microservicio de visión, se desarrolló el sistema de gestión de parqueaderos y las interfaces de usuario.

**III-D1. Backend de gestión (.NET):** El backend se implementó utilizando .NET, siguiendo una organización

por capas que separa la lógica de negocio, el acceso a datos y la exposición de la API. Entre los conceptos trabajados en esta fase se encuentran:

- **Modelado de entidades:** diseño de modelos para representar vehículos, usuarios, parqueaderos, cámaras, visitas (entradas y salidas) y tarifas. Este modelado incluyó la definición de relaciones entre entidades (por ejemplo, un vehículo puede tener múltiples visitas, cada visita está asociada a un parqueadero y a una cámara de entrada).
- **API REST:** implementación de controladores para registrar y consultar las visitas, gestionar la configuración de parqueaderos y cámaras, y exponer los datos necesarios para las interfaces web y móviles.
- **Consumo de eventos desde Kafka:** desarrollo de un componente que se suscribe al tópico de detecciones del microservicio de visión, interpreta los eventos y actualiza la base de datos con nuevas entradas o salidas, según las reglas de negocio definidas.
- **Registro de eventos y auditoría:** incorporación de mecanismos de registro (*logging*) para dejar trazas de las detecciones procesadas, las decisiones de apertura o cierre de visitas y los posibles errores, facilitando el análisis posterior.

El backend se planteó como la “fuente de verdad” respecto al estado del parqueadero, de modo que cualquier decisión de negocio (por ejemplo, calcular el tiempo de permanencia o el monto a pagar) se centraliza en este componente.

**III-D2. Interfaces web y móviles:** Sobre el backend se construyeron dos tipos de interfaces:

- **Portal web (Angular):** orientado al personal operativo y a la administración del parqueadero. Permite visualizar en tiempo cercano al real las detecciones de matrículas, consultar el estado de ocupación de las plazas, revisar el historial de visitas y gestionar la configuración del sistema (cámaras, tarifas, usuarios, etc.).
- **Aplicación móvil:** dirigida al usuario final (propietario del vehículo), con el objetivo de que pueda consultar sus vehículos registrados, el historial de accesos al parqueadero y los saldos asociados. Esta aplicación actúa principalmente como cliente de consulta sobre la API del backend.

La comunicación en tiempo cercano al real entre el backend y las interfaces se implementó mediante SignalR, que permite establecer canales bidireccionales y enviar notificaciones a los clientes cuando se registra una nueva detección o se actualiza el estado de una visita. Se trabajó en la gestión de estos canales y en la actualización dinámica de las vistas del frontend.

### III-E. Fase 5: Integración y pruebas en entorno controlado

Una vez implementados los componentes principales, se llevó a cabo una fase de integración en un entorno de

pruebas. Para ello se utilizaron contenedores Docker y archivos `docker-compose` que facilitan el despliegue de los servicios (microservicio de visión, Kafka, backend y frontends) en una misma máquina o en una red local.

Las pruebas se realizaron inicialmente con flujos de video de prueba y, posteriormente, con cámaras IP en un entorno controlado que simula el acceso a un parqueadero. Los criterios observados incluyeron:

- **Correctitud funcional:** verificación de que, al pasar un vehículo por el campo de visión de la cámara, el sistema detecta la matrícula, genera el evento correspondiente y registra la visita en la base de datos.
- **Consistencia de la información:** comprobación de que las matrículas reconocidas se mantienen coherentes a lo largo de la visita, evitando registros duplicados o inconsistentes.
- **Latencia observable:** estimación del tiempo transcurrido entre la captura del fotograma, el procesamiento en el microservicio de visión, la llegada del evento al backend y la actualización de las interfaces de usuario.

Dado que el proyecto tiene un alcance de trabajo de grado y se desarrolló con recursos limitados, no se realizó una evaluación estadística exhaustiva con grandes conjuntos de datos ni en condiciones ambientales extremas. Sin embargo, las pruebas en el entorno controlado permitieron comprobar la coherencia de la arquitectura y obtener información cualitativa sobre su comportamiento.

### III-F. Fase 6: Evaluación, limitaciones y oportunidades de ampliación

La última fase de la metodología consiste en analizar críticamente los resultados obtenidos y en identificar las principales limitaciones y oportunidades de ampliación del sistema.

Entre las limitaciones observadas se encuentran:

- La ausencia de una evaluación cuantitativa sistemática de la precisión del reconocimiento en diferentes condiciones ambientales.
- El uso de un conjunto limitado de cámaras y escenarios de prueba, que restringe la generalización de los resultados.
- La falta de módulos de analítica avanzada que exploren los datos históricos de visitas para generar indicadores de ocupación, patrones horarios o recomendaciones de mejora en la operación.

Como oportunidades de ampliación coherentes con el contexto académico de ANPR-VISION se plantean, entre otras:

- Extender la arquitectura para soportar múltiples parqueaderos y esquemas multiarrendatario, permitiendo gestionar desde una misma plataforma diferentes sitios físicos.
- Incorporar módulos de analítica de ocupación que, apoyados en los registros de visitas, permitan estimar

patrones de demanda, detectar horas pico y proponer ajustes en la capacidad o en las políticas de acceso.

- Explorar el despliegue del microservicio de visión en dispositivos de borde de bajo costo, reduciendo la latencia y la carga en el servidor central.
- Realizar campañas de evaluación con bases de datos públicas y con placas de diferentes regiones, con el fin de analizar el comportamiento del sistema en contextos diversos.

Estas líneas no forman parte del alcance del prototipo actual, pero se reconocen como caminos naturales de investigación y desarrollo para trabajos futuros que deseen construir sobre la arquitectura propuesta.

### III-G. Diagrama de arquitectura de componentes

Para complementar la descripción textual de la metodología y de la arquitectura, se incluye un diagrama de componentes que resume el flujo de información desde las cámaras hasta las interfaces de usuario. En él se representan las cámaras IP o archivos de prueba, el microservicio de visión en Python (con YOLOv5, EasyOCR y ByteTrack), el bus de mensajería Apache Kafka, el sistema de gestión de parqueaderos en .NET y las interfaces web y móviles que se comunican mediante SignalR.

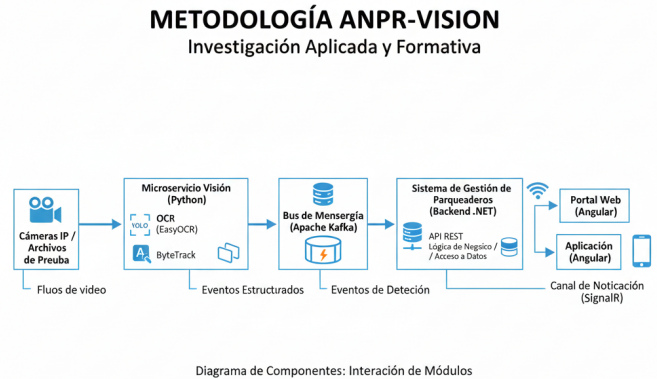


Figura 1: Arquitectura lógica de ANPR-VISION: flujo de información desde las fuentes de video hasta las interfaces web y móvil.

## IV. IMPLEMENTACIÓN DEL SISTEMA ANPR-VISION

A partir de la metodología descrita en la Sección ??, esta sección detalla cómo se materializó la arquitectura propuesta en un prototipo funcional. La implementación de ANPR-VISION se estructura alrededor de cuatro bloques

principales: (i) el microservicio de visión desarrollado en Python; (ii) el bus de mensajería basado en Apache Kafka; (iii) el sistema de gestión de parqueaderos implementado en .NET; y (iv) las interfaces web y móviles construidas con Angular. Adicionalmente, se describe la organización de la persistencia de datos y el esquema de despliegue del prototipo.

#### IV-A. Visión general de la arquitectura implementada

La Figura ?? resume la interacción entre los componentes del sistema. En el extremo de adquisición se ubican las cámaras IP o fuentes de video de prueba, cuyo flujo es procesado por el microservicio de visión en Python. Este microservicio ejecuta el modelo de detección basado en YOLOv5, la etapa de OCR con EasyOCR y el seguimiento de objetos con ByteTrack, y produce eventos estructurados que describen las detecciones de matrículas y vehículos.

Dichos eventos se publican en un tópico de Apache Kafka que actúa como intermediario entre la capa de visión y el sistema de gestión de parqueaderos. El backend .NET consume los eventos, aplica las reglas de negocio para registrar entradas y salidas de vehículos y actualiza la base de datos. Finalmente, el portal web y la aplicación móvil consumen la API del backend y reciben notificaciones en tiempo cercano al real mediante SignalR.

En las subsecciones siguientes se detalla la implementación de cada uno de estos componentes.

#### IV-B. Microservicio de visión en Python

El microservicio de visión constituye el núcleo de la parte ANPR del sistema. Se implementó en Python, aprovechando el ecosistema de bibliotecas disponibles para visión por computador y aprendizaje profundo. De manera general, el microservicio se organizó en módulos internos, cada uno con una responsabilidad bien delimitada:

- **Gestor de cámaras:** mantiene la configuración de las fuentes de video (direcciones RTSP de cámaras IP o rutas de archivos de prueba), controla la apertura y cierre de los flujos y gestiona parámetros como la tasa de lectura de fotogramas.
- **Módulo de detección:** encapsula el modelo de la familia *YOLOv5* utilizado para detectar vehículos y, cuando el entrenamiento lo permite, regiones de matrículas. Este módulo se encarga de cargar el modelo en memoria, ejecutar la inferencia sobre cada fotograma y devolver las cajas delimitadoras (*bounding boxes*) con sus respectivas etiquetas y niveles de confianza.
- **Módulo de localización de placas:** cuando la detección principal identifica solo al vehículo, este módulo aplica reglas geométricas (por ejemplo, búsqueda de regiones con proporciones típicas de placas en la parte frontal o trasera del vehículo) o modelos secundarios especializados para delimitar con mayor precisión la región de la matrícula.
- **Módulo OCR:** utiliza EasyOCR para leer los caracteres dentro de la región de la placa. Antes de llamar

al OCR, se aplican transformaciones básicas a la imagen (escala de grises, ecualización de histograma, corrección de inclinación moderada) con el fin de mejorar la nitidez del texto.

- **Módulo de seguimiento:** implementa el algoritmo ByteTrack para asociar detecciones entre fotogramas consecutivos. A cada vehículo se le asigna un identificador de seguimiento y se mantiene un historial de posiciones, lo que permite determinar cuándo un vehículo entra y sale del campo de visión de una cámara.
- **Ensamblador de eventos:** a partir de la información entregada por los módulos anteriores (identificador de seguimiento, placa reconocida, cámara, instante de tiempo, confianza, etc.), construye mensajes estructurados con un formato JSON definido para el sistema.
- **Productor Kafka:** se encarga de publicar los eventos en el tópico correspondiente de Apache Kafka, gestionando la conexión con el clúster, los reintentos en caso de fallo y la serialización de los mensajes.

La lógica principal del microservicio se implementó como un ciclo que, para cada cámara configurada, extrae fotogramas a una frecuencia configurable, ejecuta la canalización de detección-OCR-seguimiento y, cuando se detecta un cambio relevante en el estado (por ejemplo, la aparición de un nuevo identificador de seguimiento con una placa válida), genera un evento de detección.

Para facilitar el monitoreo y la depuración, se incluyeron registros de log en puntos clave del proceso (inicio de procesamiento de cámara, detección exitosa, fallos de conexión, tiempos de inferencia, etc.). Estos registros permiten identificar cuellos de botella y analizar el comportamiento del sistema en diferentes configuraciones.

Aunque el proyecto se enfocó en alcanzar un funcionamiento estable en un entorno controlado, la arquitectura del microservicio deja abiertas varias posibilidades de mejora, como incorporar modelos entrenados específicamente para placas colombianas, utilizar técnicas de aumento de datos para robustecer el OCR o integrar mecanismos de autocalibración de cámaras.

#### IV-C. Bus de mensajería con Apache Kafka

La comunicación entre el microservicio de visión y el sistema de gestión de parqueaderos se implementó mediante Apache Kafka. Se definió un tópico principal, por ejemplo `anpr.plate-detections`, en el cual el microservicio publica los eventos generados. Cada evento contiene, entre otros campos:

- Identificador de la detección o del seguimiento.
- Matrícula reconocida.
- Identificador de la cámara y del parqueadero.
- Marca de tiempo de la detección.
- Confianza o probabilidad asociada al reconocimiento.
- Tipo de evento (por ejemplo, detección inicial, actualización, posible salida).



El formato elegido para los mensajes es JSON, lo que facilita su interpretación por parte de servicios en distintos lenguajes. Para el prototipo, se configuró un solo *broker* de Kafka, suficiente para el volumen de eventos esperado en el entorno de pruebas. Sin embargo, la arquitectura es compatible con la expansión a clústeres de múltiples nodos en escenarios de mayor escala.

En el lado del microservicio, el productor Kafka se implementó utilizando una librería de cliente para Python. En el lado del backend, se utilizó un consumidor Kafka en .NET que se ejecuta como servicio en segundo plano y que se encarga de leer continuamente los mensajes del tópico, procesarlos y actualizar la base de datos.

El uso de Kafka aporta varias ventajas:

- **Desacoplamiento:** la capa de visión y el backend de gestión pueden evolucionar de forma independiente, siempre que respeten el contrato del formato de mensajes.
- **Reprocesamiento:** los mensajes quedan almacenados en el *log* de Kafka durante un tiempo configurado, lo que permite repetir pruebas o reejecutar el procesamiento en caso de cambios en la lógica de negocio.
- **Escalabilidad:** es posible añadir nuevos consumidores (por ejemplo, módulos de analítica avanzada) sin modificar el microservicio de visión.

#### IV-D. Sistema de gestión de parqueaderos en .NET

El sistema de gestión de parqueaderos se implementó como una API en .NET, organizada en capas para separar responsabilidades. De manera simplificada, se distinguen:

- **Capa de entidades:** contiene las clases que representan los objetos del dominio, como *Vehiculo*, *Visita*, *Parqueadero*, *Camara*, *Tarifa*, *Usuario* y *Notificacion*. Estas clases se mapean a tablas en la base de datos relacional.
- **Capa de datos:** implementa los repositorios y el contexto de acceso a datos, utilizando un ORM para interactuar con la base de datos (por ejemplo, Entity Framework Core).
- **Capa de negocio:** contiene los servicios que encapsulan la lógica de negocio, como el registro de visitas, la validación de matrículas, el cálculo de tiempos de permanencia y tarifas, y la generación de notificaciones.
- **Capa de presentación (API):** expone controladores REST que permiten a las interfaces web y móviles consultar la información del sistema y, en algunos casos, invocar operaciones de gestión.

Un componente central en esta capa es el *gestor de detecciones* (*VehicleDetectionManager*, en la implementación concreta), que actúa como orquestador entre el consumidor de Kafka y los servicios de negocio. Su funcionamiento puede resumirse en los siguientes pasos:

1. Recibe un mensaje desde el tópico de detecciones de Kafka.

2. Verifica la validez del evento (por ejemplo, que la matrícula tenga un formato aceptable y que la confianza supere un umbral mínimo).
3. Consulta si existe un vehículo registrado con esa placa; si no lo hay, puede crear un registro provisional o marcar la detección para revisión según las reglas definidas.
4. Determina, a partir del estado actual del sistema, si la detección corresponde al inicio de una nueva visita o a la actualización de una visita en curso (por ejemplo, posible salida).
5. Actualiza la base de datos creando o modificando registros de visitas, calculando el tiempo de permanencia y dejando trazas de auditoría.
6. En los casos relevantes, solicita al servicio de notificaciones la emisión de un mensaje hacia los clientes conectados (por ejemplo, para informar al operador de una nueva entrada).

En términos de persistencia, la base de datos almacena tanto información maestra (parqueaderos, cámaras, usuarios, vehículos registrados) como información transaccional (visitas, cobros, notificaciones). El diseño de las tablas y relaciones se realizó de manera coherente con los diagramas de entidad-relación elaborados en fases previas.

#### IV-E. Interfaces web y aplicación móvil

Sobre la API del backend se construyeron dos tipos de interfaces que materializan el uso del sistema por parte de los distintos actores.

*IV-E1. Portal web para operadores y administradores:* El portal web se implementó con Angular. Su estructura se organiza en módulos funcionales, entre los que destacan:

- **Módulo de monitoreo:** presenta en una vista principal el estado de las cámaras y las detecciones recientes, así como un historial de entradas y salidas. Se integra con SignalR para recibir notificaciones en tiempo cercano al real cuando se detecta una nueva matrícula.
- **Módulo de gestión de parqueaderos:** permite administrar la información de los parqueaderos, plazas, cámaras y tarifas. Incluye formularios para crear y editar estos elementos y tablas para su consulta.
- **Módulo de vehículos y usuarios:** ofrece funciones básicas para gestionar vehículos registrados y usuarios del sistema, lo que facilita vincular visitas a propietarios específicos en escenarios donde se requiera esta información.
- **Módulo de reportes:** proporciona consultas sobre el historial de visitas y métricas simples de uso, como el número de entradas por día o el tiempo promedio de permanencia.

Visualmente, el portal se diseñó con un enfoque práctico, priorizando la presentación clara de información relevante para la operación diaria del parqueadero.

*IV-E2. Aplicación móvil para usuarios finales:* La aplicación móvil, también desarrollada con tecnologías basadas en Angular, está orientada al usuario final, entendido

como el propietario o conductor de un vehículo que utiliza con frecuencia el parqueadero. Entre sus funcionalidades básicas se incluyen:

- Visualización de los vehículos asociados a la cuenta del usuario.
- Consulta del historial de accesos (entradas y salidas) de cada vehículo al parqueadero.
- Visualización de saldos o montos asociados a las visitas, cuando el modelo de negocio lo requiera.

La aplicación consume la API del backend mediante llamadas HTTP autenticadas. En esta etapa del proyecto, su rol principal es de consulta; no se implementaron aún funciones avanzadas como reserva de plazas o pagos en línea, que se consideran posibles ampliaciones futuras.

#### IV-F. Persistencia de datos y modelo relacional

La persistencia del sistema se implementó mediante una base de datos relacional compartida por los distintos componentes del backend. A nivel lógico, el modelo es el mismo para todos los entornos; lo que cambia es la infraestructura sobre la que se despliega.

En los entornos de desarrollo y prueba se utilizó una base de datos relacional ejecutada en contenedor Docker, lo que facilita la recreación del entorno desde cero y la realización de pruebas controladas. En el entorno de producción, en cambio, la base de datos se desplegó en un servicio administrado en la nube (Amazon RDS), aprovechando sus capacidades de respaldo, monitorización y gestión de recursos.

El modelo relacional incluye, entre otras, las siguientes tablas:

- **Parqueadero:** almacena información básica de cada parqueadero (identificador, nombre, dirección, parámetros de configuración).
- **Camara:** registra las cámaras asociadas a un parqueadero, incluyendo la URL de conexión, el tipo de flujo y el parqueadero al que pertenecen.
- **Vehículo:** contiene los datos de los vehículos (placa, tipo, posibles asociaciones con usuarios).
- **RegistroVehículo:** representa cada ingreso de un vehículo al parqueadero, con campos como hora de entrada, hora de salida, identificador de la cámara, tiempo de permanencia y monto calculado (cuando aplica).
- **Tarifa:** define las reglas de cobro (por ejemplo, valor por minuto u hora, franjas horarias especiales o tarifas diferenciadas por tipo de usuario).
- **Usuario:** almacena información de los usuarios del sistema (operadores, administradores, usuarios finales) y sus credenciales de acceso.
- **Notificacion:** registra las notificaciones generadas por eventos relevantes (detecciones, cierres de visitas, alertas operativas).

Las relaciones entre tablas (por ejemplo, **Parqueadero-Camara**, **Vehículo-Visita**) se diseñaron para garantizar

integridad referencial y facilitar consultas frecuentes, como el historial de visitas de un vehículo, el nivel de ocupación de un parqueadero en un intervalo de tiempo o las visitas asociadas a un usuario registrado. El mismo esquema se replica en los cuatro entornos definidos (desarrollo, QA, *staging* y producción), variando únicamente las credenciales y parámetros de conexión según cada caso.

#### IV-G. Despliegue del prototipo en múltiples entornos

El despliegue de ANPR-VISION se organizó en cuatro entornos diferenciados: desarrollo, QA, *staging* y producción. Esta separación permite validar de manera progresiva los cambios antes de que lleguen al entorno expuesto a los usuarios finales.

En todos los entornos se utilizaron contenedores Docker y archivos `docker-compose` específicos por ambiente, que orquestan los servicios necesarios. De forma general, cada entorno incluye:

- Un contenedor para el backend de gestión de parqueaderos (.NET).
- Un contenedor para el portal web Angular.
- Un contenedor para la base de datos relacional (en desarrollo y QA) o una conexión hacia la instancia RDS en la nube (en producción).
- Contenedores adicionales para Apache Kafka y, cuando corresponde, para el microservicio de visión en Python, utilizados principalmente en los entornos de pruebas y demostración.

Sobre esta base se configuraron canalizaciones de integración y despliegue continuo (*pipelines* CI/CD) utilizando Jenkins. Para el backend y el portal web, cada cambio aceptado en el repositorio dispara un proceso automatizado que:

1. Compila el proyecto y ejecuta las pruebas básicas.
2. Construye la imagen Docker correspondiente, etiquetada según el entorno.
3. Despliega la nueva versión utilizando el archivo `docker-compose` del entorno seleccionado.

En el entorno de producción, el despliegue se realiza sobre una máquina virtual en Amazon Web Services (AWS) que hospeda los contenedores del backend, el frontend y los servicios auxiliares necesarios. Jenkins se conecta de forma remota al servidor, actualiza las imágenes y reinicia los contenedores con la nueva versión, manteniendo la configuración de red y seguridad definida (por ejemplo, acceso HTTPS a través de un proxy inverso y reglas de firewall).

El microservicio de visión, por su parte, se empaquetó también en contenedor Docker y se desplegó principalmente en entornos de desarrollo y prueba para validar la arquitectura completa. Su despliegue en producción se considera una línea de trabajo futuro, una vez se cuente con una evaluación más amplia del comportamiento del modelo en condiciones reales.

Este esquema de despliegue no pretende cubrir todos los aspectos de una solución empresarial de alta disponibilidad (como balanceo de carga o redundancia geográfica), pero resulta suficiente para validar la coherencia de la arquitectura, practicar el uso de herramientas de contenedores y CI/CD, y facilitar la reproducción del entorno en otros contextos académicos.

En conjunto, la implementación de ANPR-VISION pone en práctica la arquitectura propuesta y permite observar de forma tangible cómo interactúan la capa de visión, la mensajería distribuida, la lógica de negocio y las interfaces de usuario en un escenario de despliegue realista pero controlado. Sobre esta base se realiza, en la siguiente sección, una discusión de los resultados obtenidos y de las lecciones aprendidas durante el desarrollo del proyecto.

Para complementar la descripción textual de la implementación, en la Figura ?? se presenta una vista detallada de la arquitectura de ANPR-VISION. En ella se observa la canalización completa desde la captura del flujo de video en la cámara IP, el procesamiento en el microservicio de visión (detección con YOLOv5, localización de la placa, OCR con EasyOCR y seguimiento con ByteTrack), la publicación del evento estructurado en el bus de mensajería Apache Kafka y, finalmente, el consumo de dichos eventos por el sistema de gestión en .NET, que actualiza la base de datos y expone la información a las interfaces web y móviles mediante una API REST y notificaciones con SignalR.

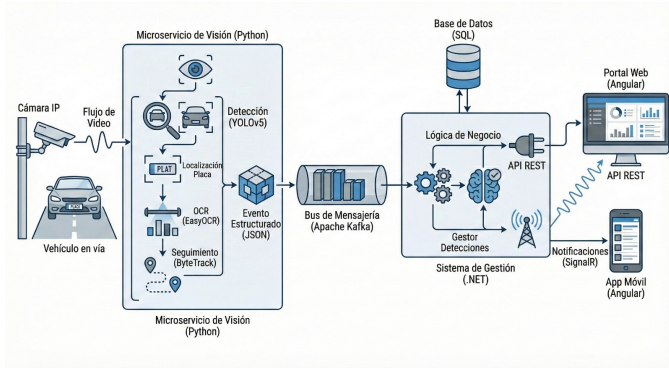


Figura 2: Arquitectura detallada de ANPR-VISION: flujo de información desde la cámara IP hasta el backend de gestión y las interfaces de usuario.

## V. RESULTADOS

Esta sección presenta los resultados obtenidos a partir de la implementación del prototipo ANPR-VISION descrito en la Sección ?. Dado que el proyecto se desarrolló como trabajo de grado en el marco del SENA, con recursos limitados y un énfasis formativo, los resultados se centran en la validación funcional de la arquitectura, en observaciones de rendimiento en un entorno controlado y en la experiencia de despliegue en múltiples entornos, más que en una evaluación estadística exhaustiva.

### V-A. Entorno de pruebas y configuración experimental

Las pruebas se realizaron en un entorno controlado que simula el acceso a un parqueadero. Por razones de tiempo y de disponibilidad de hardware, en lugar de utilizar una cámara IP dedicada se empleó un *smartphone* Android configurado con la aplicación *IP Webcam*. Esta aplicación convierte el teléfono en una cámara de red, exponiendo el flujo de video a través de la red local mediante una URL de tipo HTTP/RTSP, de forma equivalente a una cámara IP convencional.

El montaje experimental incluyó:

- Un teléfono móvil con la aplicación *IP Webcam* actuando como cámara IP, ubicado a la entrada de un espacio de parqueo y orientado de manera que capturara el frontal de los vehículos a una distancia y ángulo moderados.
- Un servidor local (o máquina virtual) ejecutando los contenedores de Docker correspondientes al microservicio de visión, al *broker* de Apache Kafka, al backend .NET y al portal web Angular.
- La base de datos relacional desplegada como contenedor en los entornos de desarrollo y pruebas, y una instancia adicional en Amazon RDS para validar la conectividad con el entorno de producción.

En la práctica, el microservicio de visión consume la URL de video generada por *IP Webcam* igual que consumiría el flujo de una cámara IP física, por lo que la forma en que se configuró el experimento no modifica la arquitectura ni el código del sistema, sino únicamente la fuente de video utilizada.

Se realizaron pruebas tanto con flujos de video en tiempo real provenientes del teléfono (a través de *IP Webcam*) como con archivos de video previamente grabados, que permitieron reproducir varias veces las mismas secuencias de vehículos para observar el comportamiento del sistema bajo condiciones equivalentes.

En cada sesión de prueba se registraron:

- El número de vehículos que pasaron por el campo de visión de la cámara simulada y sus matrículas.
- Las detecciones generadas por el microservicio de visión.
- Los registros de visitas creados en la base de datos.
- La latencia aproximada entre la captura de la imagen y la visualización de la detección en el portal web.

Aunque las pruebas se realizaron con un teléfono actuando como cámara IP, la forma en que se diseñó la arquitectura (uso de direcciones RTSP/HTTP como fuentes de video) permite sustituir esta fuente por cámaras IP reales sin cambios estructurales en el sistema, limitándose a ajustar las URL y parámetros de conexión.

### V-B. Validación funcional de la arquitectura

Desde el punto de vista funcional, el objetivo principal era verificar que la canalización completa —desde la captura del flujo de video hasta el registro de visitas y

la notificación en las interfaces— se comportara según lo especificado.

De manera general, las pruebas mostraron que:

- El microservicio de visión fue capaz de detectar vehículos y localizar la región de la placa en la mayoría de los fotogramas en los que la matrícula era claramente visible en el flujo generado por *IP Webcam*.
- En las secuencias de prueba, el OCR basado en EasyOCR logró recuperar correctamente la cadena de caracteres de la matrícula en un porcentaje alto de los casos, especialmente cuando la placa no presentaba suciedad notable ni reflejos intensos.
- El algoritmo de seguimiento ByteTrack permitió mantener la identidad del vehículo durante el tiempo que permanecía en el campo de visión, evitando que pequeñas variaciones en la detección generaran múltiples visitas falsas.
- Por cada vehículo que cruzó el punto de acceso, el backend .NET registró al menos una visita en la base de datos, asociando correctamente la matrícula detectada, la cámara (en este caso, el teléfono configurado como cámara IP), la hora de entrada y, cuando se completó el ciclo, la hora de salida.
- El portal web Angular actualizó su vista de monitoreo pocos instantes después de cada detección, mostrando al operador la nueva visita en la lista de eventos recientes.

En términos generales, el comportamiento observado confirma que la arquitectura propuesta es funcional: los componentes se comunican correctamente a través de Kafka, el backend procesa los eventos como se esperaba y las interfaces reflejan el estado del sistema de forma coherente, independientemente de que la fuente de video sea una cámara IP dedicada o un teléfono configurado como tal.

#### V-C. Observaciones sobre rendimiento y latencia

Aunque no se realizó una medición de rendimiento de tipo industrial, sí se recopilaron observaciones cualitativas y algunas estimaciones aproximadas que permiten tener una idea del comportamiento del prototipo en el entorno de pruebas.

En las secuencias evaluadas se observó que:

- El tiempo de procesamiento por fotograma en el microservicio de visión se mantuvo dentro de un orden de magnitud compatible con aplicaciones en tiempo casi real, tomando en cuenta la resolución del video y el hardware disponible. Para reducir la carga, se configuró el sistema para procesar solo una fracción de los fotogramas (por ejemplo, uno de cada varios), lo que resultó suficiente para capturar las matrículas de vehículos que pasaban a baja velocidad frente a la cámara simulada.
- La latencia observable entre el momento en que un vehículo entraba en el campo de visión y el momento en que la detección aparecía en el portal web fue, en

la mayoría de los casos, de unos pocos segundos. Esta latencia incluye el tiempo de captura, inferencia con YOLOv5, OCR, envío del evento a Kafka, procesamiento en el backend y actualización de la interfaz vía SignalR.

- No se detectaron cuellos de botella significativos en Kafka en el escenario de prueba, dado el volumen relativamente bajo de eventos. El *broker* funcionó de manera estable y permitió repetir las pruebas utilizando los mismos mensajes almacenados.

Cuando se trabajó con videos de prueba que incluían condiciones menos favorables (movimiento más rápido del vehículo, variaciones de iluminación o pequeñas oclusiones), se observó un aumento de errores de reconocimiento (por ejemplo, confusiones entre letras y números visualmente similares). Estos casos pusieron de manifiesto la necesidad de contar con datos de entrenamiento específicos para el tipo de placas y condiciones locales si se quiere alcanzar niveles de precisión más altos en un despliegue real, ya sea con cámaras IP físicas o con dispositivos móviles actuando como tales.

La Tabla ?? resume, de manera estructurada, los principales aspectos observados. Los valores numéricos pueden ajustarse con datos medidos más precisos en caso de que se realicen campañas de pruebas adicionales.

Tabla I: Resumen cualitativo de resultados observados en el entorno de pruebas.

| Aspecto                            | evaluado              | Observaciones principales                                                                                                                                                                                             |
|------------------------------------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Detección                          | de vehículos y placas | El modelo YOLOv5 detectó la mayoría de los vehículos con la placa visible en videos de prueba grabados en condiciones de iluminación moderada. Se observaron fallos en casos de reflejos fuertes o placas muy sucias. |
| Reconocimiento de caracteres (OCR) |                       | EasyOCR recuperó correctamente la matrícula en un porcentaje alto de los casos con buena calidad de imagen. Las confusiones más frecuentes se dieron entre caracteres similares (por ejemplo, 0/0, B/8).              |
| Seguimiento                        | de vehículos          | ByteTrack permitió asociar detecciones consecutivas al mismo vehículo, reduciendo lecturas duplicadas y dando soporte a la estimación de entrada y salida.                                                            |
| Latencia extremo a extremo         |                       | La actualización de la interfaz web se produjo generalmente dentro de unos pocos segundos después de la captura del fotograma, suficiente para el uso operativo en el entorno de prueba.                              |
| Estabilidad                        | de la arquitectura    | Los servicios desplegados en Docker (microservicio de visión, Kafka, backend y frontend) funcionaron de manera estable durante sesiones prolongadas de prueba, sin caídas inesperadas.                                |

#### V-D. Experiencia de despliegue en múltiples entornos

Además del comportamiento funcional, se evaluó la capacidad del sistema para ser desplegado y actualizado en los distintos entornos definidos (desarrollo, QA, *staging* y producción). En este aspecto, los resultados fueron los siguientes:

- La definición de archivos `docker-compose` por entorno permitió reproducir de manera consistente la pila de servicios (backend, frontend, base de datos y servicios auxiliares) en cada servidor, reduciendo la variabilidad entre instalaciones.
- La integración con Jenkins facilitó la automatización de compilación, construcción de imágenes Docker y despliegue de nuevas versiones del backend y del portal web. Cada cambio en el repositorio activaba un *pipeline* que, tras pasar las etapas básicas de verificación, actualizaba el entorno correspondiente.
- En el caso particular de producción, la combinación de una máquina virtual en AWS para los contenedores y una instancia de base de datos en Amazon RDS permitió simular un escenario de despliegue cercano a una solución real, manteniendo a la vez el control necesario para fines educativos.
- El microservicio de visión se desplegó principalmente en entornos de desarrollo y prueba, donde se validó la comunicación extremo a extremo. Su despliegue permanente en producción se dejó como trabajo futuro, condicionado a una evaluación más amplia de comportamiento y consumo de recursos.

Desde el punto de vista formativo, la existencia de estos entornos y *pipelines* contribuyó a que el equipo de desarrollo experimentara con prácticas de despliegue automatizado y de gestión de configuraciones por ambiente, aun cuando el foco principal del trabajo recayera en la arquitectura ANPR.

#### V-E. Síntesis de hallazgos

En síntesis, los resultados obtenidos permiten extraer los siguientes puntos clave:

- La arquitectura propuesta es técnicamente viable: el flujo completo desde la captura de video (ya sea desde una cámara IP dedicada o desde un teléfono configurado con *IP Webcam*) hasta la visualización en las interfaces se ejecuta de manera coherente, y los componentes se encuentran adecuadamente desacoplados gracias al uso de Apache Kafka.
- El prototipo ANPR-VISION cumple con el objetivo de servir como plataforma educativa y de experimentación, al integrar conceptos de visión por computador, microservicios, mensajería distribuida y desarrollo web y móvil en un caso de uso concreto.
- El desempeño observado en el entorno de pruebas es adecuado para un escenario controlado, pero se identifican limitaciones en la precisión del reconocimiento bajo condiciones adversas y en la ausencia de una evaluación cuantitativa más amplia.
- El esquema de despliegue basado en contenedores y *pipelines* de Jenkins demostró ser útil para gestionar múltiples entornos y automatizar actualizaciones, acercando el proyecto a prácticas contemporáneas de desarrollo de software.

Estas observaciones sirven de base para la discusión crítica que se presenta en la siguiente sección, en la que se analizan las fortalezas y debilidades de la propuesta y se plantean posibles líneas de evolución para trabajos futuros.

## VI. DISCUSIÓN

Los resultados presentados en la Sección ?? permiten reflexionar sobre el alcance real de ANPR-VISION, sus aportes frente a la literatura revisada y las limitaciones que condicionan su uso en escenarios de producción. En esta sección se discuten estos aspectos desde cuatro ejes: (i) coherencia de la arquitectura propuesta, (ii) pertinencia del uso de tecnologías abiertas en el contexto del SENA, (iii) desempeño observado y limitaciones técnicas, y (iv) oportunidades de extensión e implicaciones para trabajos futuros.

### VI-A. Coherencia de la arquitectura basada en microservicio de visión y mensajería distribuida

Uno de los objetivos centrales del trabajo fue demostrar que es posible estructurar un sistema ANPR para gestión de parqueaderos siguiendo una arquitectura modular, en la que la capa de visión por computador se encuentre desacoplada de la lógica de negocio. En este sentido, la implementación de un microservicio de visión en Python conectado al backend .NET mediante Apache Kafka mostró ser una estrategia coherente.

Frente a las soluciones descritas en la literatura sobre parqueo inteligente y ciudades inteligentes, donde con frecuencia se presenta el sistema como un bloque monolítico o se omiten detalles de integración `streetsavvy_2024`, `campussense_2017`, `realtime_parking_two_cameras_2024`, ANPR-VISION explicita la separación entre: (i) adquisición y procesamiento de video, (ii) transporte de eventos, (iii) gestión de datos y reglas de negocio, y (iv) presentación en interfaces. Esta separación facilita razonar sobre cada componente de manera independiente, probarlos por separado y, en perspectiva, reemplazarlos sin rediseñar el sistema completo.

Un aspecto particular del prototipo es que, por razones de disponibilidad de hardware, las pruebas se realizaron utilizando un *smartphone* con la aplicación *IP Webcam* actuando como cámara IP. Sin embargo, desde el punto de vista arquitectónico, el microservicio de visión consume una URL de flujo de video (HTTP/RTSP) sin importar si la fuente es un teléfono o una cámara IP dedicada. Esto refuerza la idea de desacoplamiento: el sistema está preparado para trabajar con cámaras IP reales simplemente cambiando la configuración de las direcciones de origen, sin modificaciones en la lógica interna.

El uso de Kafka como *log* distribuido introduce, además, capacidades de reprocesamiento de eventos y de suscripción de nuevos consumidores (por ejemplo, módulos de analítica avanzada) sin afectar al microservicio de visión.

Si bien en el entorno de pruebas el volumen de datos es reducido y podría haberse utilizado una comunicación directa mediante HTTP, la elección de Kafka se justifica como decisión de diseño que prepara el sistema para escenarios de mayor escala y sirve como experiencia formativa en arquitecturas orientadas a eventos.

No obstante, esta misma decisión introduce complejidad adicional en la configuración y en el despliegue, especialmente para equipos en formación. La necesidad de configurar tópicos, gestionar el *broker* y entender la semántica de consumo puede representar una barrera inicial. En el proyecto, esta complejidad se maneja parcialmente mediante el uso de contenedores y archivos `docker-compose`, pero sigue siendo un aspecto a considerar en procesos de transferencia a otros grupos de estudiantes.

#### *VI-B. Uso de herramientas de código abierto y contexto educativo*

Otro eje relevante de discusión es la pertinencia del uso de herramientas de código abierto en un trabajo de grado del SENA. La combinación de YOLOv5, EasyOCR, ByteTrack, .NET, Angular, Kafka y Docker conforma un conjunto tecnológico heterogéneo pero accesible, que permite abordar un problema realista sin incurrir en costos de licenciamiento.

En comparación con soluciones comerciales de ANPR y de aparcamiento inteligente, que suelen ser cajas negras difíciles de inspeccionar o modificar, ANPR-VISION ofrece transparencia sobre cómo se implementa cada etapa del proceso. Desde la perspectiva educativa, esto resulta especialmente valioso: los aprendices tienen la posibilidad de examinar el código del microservicio de visión, ajustar parámetros del modelo, modificar la lógica de negocio del backend o extender las vistas del frontend, lo que contribuye a una comprensión integral del sistema.

Asimismo, el hecho de desplegar la base de datos de producción en Amazon RDS y los servicios en una máquina virtual en AWS, utilizando canalizaciones de Jenkins, acerca el proyecto a prácticas de despliegue contemporáneas sin dejar de ser un entorno controlado. Para un trabajo de grado de nivel tecnológico, esta combinación de tecnologías representa un equilibrio entre ambición técnica y viabilidad de implementación.

Sin embargo, el uso de múltiples tecnologías implica también una curva de aprendizaje considerable. La integración de Python para visión, C# para backend, TypeScript para frontend, además de Kafka, Docker y Jenkins, puede resultar retadora para equipos pequeños. En el desarrollo de ANPR-VISION, esta complejidad se abordó de manera incremental, pero constituye una limitación a tener en cuenta si se pretende replicar el proyecto en contextos con menos tiempo disponible o con menor experiencia previa.

#### *VI-C. Desempeño del prototipo y limitaciones técnicas*

Desde el punto de vista del desempeño, los resultados obtenidos indican que el prototipo es capaz de operar en

tiempo casi real en un entorno controlado, con latencias de unos pocos segundos desde la captura de la imagen hasta la visualización de la detección en el portal web. Este comportamiento es consistente con otros trabajos que utilizan modelos de la familia YOLO para detectar vehículos y matrículas en aplicaciones de control de acceso **deep\_anpr\_pipeline\_access\_control\_2022, yolo\_high\_precision\_lpr\_2024**.

No obstante, el sistema presenta varias limitaciones:

- La precisión del reconocimiento depende fuertemente de la calidad del video y de las condiciones de iluminación. En casos de reflejos, suciedad en la placa o variaciones extremas de ángulo, el OCR comete errores similares a los descritos en la literatura **lpr\_ai\_dl\_2025**, lo que afectaría la confiabilidad del sistema en entornos de operación reales.
- El modelo de detección no fue entrenado específicamente con placas colombianas ni con un conjunto amplio de imágenes locales, lo que reduce su capacidad para generalizar a todas las variaciones de diseño y desgaste presentes en el contexto.
- La evaluación realizada se limita a un número acotado de secuencias de prueba y no incluye un análisis estadístico sistemático con métricas estándar (precisión, *recall*, F1, etc.), tal como suele encontrarse en estudios centrados en algoritmos **anpr\_pakistan\_2020**. Esta ausencia se debe a la orientación aplicada y formativa del proyecto, pero limita la comparabilidad directa con otros trabajos.
- El entorno de pruebas utilizó un teléfono con *IP Webcam* como fuente de video, lo que, aunque es funcionalmente equivalente a una cámara IP desde el punto de vista de la arquitectura, no reproduce todas las características ópticas y de instalación de una cámara de vigilancia profesional.
- El microservicio de visión aún no se ha desplegado de manera permanente en producción, por lo que no se dispone de evidencia sobre su comportamiento continuo en condiciones de carga y variabilidad ambiental propias de un parqueadero real.

Estas limitaciones no invalidan la propuesta, pero sí acotan claramente su alcance: ANPR-VISION debe entenderse como un prototipo arquitectónico y educativo, más que como un producto listo para explotación comercial inmediata.

#### *VI-D. Oportunidades de ampliación y comparación con trabajos relacionados*

A partir de las limitaciones identificadas, se pueden delinear varias líneas de trabajo futuro que, además, se alinean con las tendencias observadas en la literatura.

En primer lugar, resulta natural avanzar hacia una evaluación cuantitativa más completa. Esto implicaría construir o adaptar un conjunto de datos de matrículas capturadas en condiciones locales y etiquetadas de manera precisa, siguiendo metodologías similares a las empleadas

en estudios que desarrollan modelos específicos para determinadas regiones [software\\_libre\\_mercosur\\_2016](#), [anpr\\_pakistan\\_2020](#). Sobre esta base, sería posible comparar diferentes configuraciones de modelo (por ejemplo, variaciones de YOLO, modelos de OCR alternativos) y justificar de forma más robusta las decisiones tecnológicas.

En segundo lugar, la arquitectura basada en Kafka abre la puerta a la incorporación de módulos de analítica avanzada y de gestión multiparqueadero. Varios trabajos de *smart parking* exploran el uso de datos históricos para optimizar la asignación de plazas, detectar vehículos mal estacionados o estimar niveles de ocupación [smart\\_parking\\_wrongly\\_parked\\_2023](#), [smart\\_parking\\_lpr\\_efficiency\\_2024](#). En ANPR-VISION, un módulo adicional podría suscribirse a los eventos de detección y generar indicadores de ocupación, patrones horarios o recomendaciones de mejora para la operación, aprovechando el modelo de datos ya establecido.

En tercer lugar, la exploración de despliegues en dispositivos de borde (por ejemplo, minicomputadores cercanos a las cámaras) se alinea con propuestas que buscan reducir la latencia y la dependencia de enlaces de red de alta calidad [realtime\\_ocr\\_wsn\\_its\\_2019](#). La separación clara entre microservicio de visión y backend de gestión favorece este tipo de experimentos, al permitir mover el procesamiento de video hacia el borde sin alterar la lógica de negocio central.

Finalmente, desde la perspectiva educativa, ANPR-VISION puede servir como base para proyectos posteriores que profundicen en aspectos específicos: optimización del modelo de visión, diseño de interfaces más avanzadas, integración con métodos de pago o incorporación de mecanismos de seguridad adicionales (listas negras, detección de anomalías, etc.), tal como se plantea en trabajos orientados a seguridad vehicular [anpr\\_security\\_bangladesh\\_2025](#).

#### VI-E. Implicaciones para la formación en análisis y desarrollo de software

Más allá de los aspectos puramente técnicos, el desarrollo de ANPR-VISION tiene implicaciones relevantes para la formación en análisis y desarrollo de software en el SENA. El proyecto permitió a los estudiantes y aprendices:

- Trabajar con un problema realista y multidisciplinar, que combina visión por computador, arquitectura de software, bases de datos, desarrollo web y móvil, y despliegue en la nube.
- Experimentar con buenas prácticas de ingeniería de software, como la separación de capas, el uso de contenedores, la gestión de entornos y la automatización de despliegues.
- Enfrentarse a decisiones de diseño con trade-offs claros (por ejemplo, complejidad de Kafka frente a sencillez de HTTP; flexibilidad de microservicios frente

a simplicidad de un monolito) y reflexionar sobre sus consecuencias.

En este sentido, el valor del proyecto trasciende los resultados de precisión del reconocimiento de matrículas. Su principal aporte se sitúa en mostrar que es posible abordar, desde un programa de formación tecnológica, la construcción de un sistema distribuido basado en tecnologías contemporáneas, documentando las decisiones clave y sus justificaciones. Este tipo de experiencias contribuye a reducir la brecha entre los contenidos curriculares y las prácticas de la industria, y ofrece a futuros grupos de estudiantes una referencia concreta sobre la cual construir nuevas iniciativas.

#### VI-F. Amenazas a la validez

Como en toda investigación aplicada, los resultados y conclusiones de este trabajo están sujetos a varias amenazas a la validez:

- **Validez interna:** algunas decisiones de configuración (por ejemplo, umbrales de confianza, tasa de fotografías procesados) se ajustaron de forma empírica durante las pruebas, sin un protocolo experimental sistemático. Esto puede sesgar las observaciones de desempeño hacia los escenarios probados.
- **Validez externa:** el entorno de pruebas, aunque realista a pequeña escala, se basa en una cámara simulada mediante *IP Webcam* y no reproduce completamente la diversidad de condiciones ambientales, tipos de dispositivos y modos de instalación propios de un sistema de vigilancia con cámaras IP profesionales. Por tanto, los resultados no pueden generalizarse sin reservas a esos contextos.
- **Validez de constructo:** el énfasis del proyecto en la arquitectura y en el prototipo funcional hace que no se hayan medido explícitamente algunas variables típicas de estudios de visión por computador (como curvas ROC o métricas por clase), lo que limita la comparación directa con trabajos centrados exclusivamente en el algoritmo.

Reconocer estas amenazas es importante para evitar conclusiones sobredimensionadas. En lugar de presentar ANPR-VISION como una solución definitiva, el proyecto se posiciona explícitamente como una arquitectura de referencia y un caso de estudio educativo, que puede ser refinado y ampliado en investigaciones posteriores.

En conjunto, la discusión anterior refuerza la idea de que el principal aporte de ANPR-VISION radica en la integración coherente de tecnologías abiertas y en la documentación detallada de una arquitectura ANPR orientada a la gestión de parqueaderos, más que en la introducción de un nuevo algoritmo de reconocimiento. Esta distinción es clave para situar adecuadamente el trabajo dentro del panorama de la literatura existente y de las posibilidades de evolución futura.

## VII. CONCLUSIONES Y TRABAJO FUTURO

### VII-A. Conclusiones

El trabajo presentado describe el diseño e implementación de ANPR-VISION, un sistema de Reconocimiento Automático de Matrículas orientado a la gestión de parqueaderos, desarrollado como trabajo de grado en el marco del Servicio Nacional de Aprendizaje (SENA). Más que proponer un nuevo algoritmo de visión por computador, el objetivo ha sido construir y documentar una arquitectura integral que integre componentes de visión, mensajería distribuida, gestión de datos y interfaces de usuario, utilizando herramientas de código abierto y prácticas actuales de ingeniería de software.

En primer lugar, se logró diseñar y materializar una arquitectura modular basada en un microservicio de visión implementado en Python, un backend de gestión desarrollado en .NET, un bus de mensajería construido sobre Apache Kafka y frontends web y móvil desarrollados con tecnologías tipo Angular. Esta organización permite desacoplar la capa de visión por computador de la lógica de negocio, favoreciendo la extensibilidad del sistema y alineándose con propuestas de arquitecturas orientadas a eventos en entornos de aparcamiento inteligente.

En segundo lugar, se implementó un prototipo funcional que integra modelos de la familia YOLOv5 para la detección de vehículos y matrículas, EasyOCR para el reconocimiento de caracteres y ByteTrack para el seguimiento de objetos, combinados en una canalización de procesamiento capaz de operar en tiempo casi real en un entorno controlado. Aunque las pruebas se realizaron utilizando un *smartphone* con la aplicación *IP Webcam* como fuente de video en lugar de cámaras IP dedicadas, la arquitectura se diseñó para consumir flujos RTSP/HTTP de forma genérica, por lo que el sistema es fácilmente adaptable a cámaras de red reales cambiando únicamente la configuración.

En tercer lugar, se demostró la viabilidad de desplegar la solución en múltiples entornos (desarrollo, QA, *staging* y producción), utilizando contenedores Docker, archivos `docker-compose` por ambiente y canalizaciones de integración y entrega continua con Jenkins. El uso de una máquina virtual en Amazon Web Services (AWS) y de una base de datos administrada en Amazon RDS permitió simular un escenario de despliegue cercano a un entorno productivo, manteniendo un control acorde con el carácter educativo del proyecto.

En cuarto lugar, las pruebas realizadas en el entorno controlado mostraron que la arquitectura propuesta es funcional: el flujo completo desde la captura del video hasta el registro de visitas y la notificación en el portal web se ejecuta de manera coherente, con latencias de unos pocos segundos y un nivel de estabilidad adecuado para sesiones prolongadas. Sin embargo, también se evidenciaron limitaciones en la precisión del reconocimiento bajo condiciones adversas (iluminación variable, reflejos,

suciedad en las placas) y en la ausencia de una evaluación cuantitativa sistemática con métricas estándar.

Finalmente, desde la perspectiva formativa, ANPR-VISION permitió a los aprendices trabajar con un problema realista que combina visión por computador, arquitectura de software, bases de datos, desarrollo web y móvil, y despliegue en la nube. El proyecto ilustra que, incluso con recursos limitados, es posible abordar tecnologías contemporáneas y producir una arquitectura documentada que sirva como referencia para futuros trabajos en el ámbito del análisis y desarrollo de software.

En conjunto, ANPR-VISION debe entenderse como una arquitectura de referencia y un caso de estudio educativo para sistemas ANPR orientados a la gestión de parqueaderos, más que como un producto listo para explotación comercial inmediata. Su principal aporte radica en la integración coherente de tecnologías abiertas, en la separación clara de responsabilidades entre componentes y en la documentación del proceso seguido.

### VII-B. Trabajo futuro

A partir de las conclusiones anteriores y de las limitaciones identificadas, se plantean diversas líneas de trabajo futuro que podrían ampliar y profundizar los resultados obtenidos:

- **Evaluación cuantitativa sistemática:** diseñar y ejecutar campañas de pruebas con conjuntos de datos más amplios, preferiblemente capturados en contextos locales (parqueaderos reales, diferentes tipos de vehículos y condiciones de iluminación). Sobre esta base, calcular métricas estándar (precisión, *recall*, F1, tasas de falsos positivos y negativos) que permitan comparar de manera rigurosa el desempeño del sistema con el de otros enfoques reportados en la literatura.
- **Adaptación del modelo a placas locales:** entrenar o ajustar modelos de detección y reconocimiento específicamente para placas colombianas (u otras regiones de interés), incorporando variaciones de diseño, desgaste y condiciones ambientales. Esto podría incluir la creación de un conjunto de datos anotado propio y la exploración de técnicas de *fine-tuning* sobre modelos preentrenados.
- **Robustez frente a condiciones adversas:** estudiar estrategias para mejorar la tolerancia a iluminación extrema, reflejos, suciedad y oclusiones parciales. Entre las posibilidades se incluyen técnicas de preprocesamiento de imagen más avanzadas, uso de filtros específicos, control de exposición en las cámaras y exploración de arquitecturas de red más robustas al ruido.
- **Despliegue del microservicio de visión en producción y en el borde:** llevar el microservicio de visión a un entorno de producción real, monitorizando su comportamiento de forma continua y analizando consumo de recursos, estabilidad y mantenimiento.



En paralelo, explorar su ejecución en dispositivos de borde (por ejemplo, minicomputadores cercanos a las cámaras), con el fin de reducir la latencia y la dependencia de enlaces de red de alta capacidad.

- **Extensión a escenarios multiparqueadero y multiarrendatario:** generalizar el modelo de datos y la lógica de negocio para soportar múltiples parqueaderos gestionados desde una misma plataforma, incluyendo mecanismos de separación lógica (*multi-tenant*) y de configuración específica por sitio (tarifas, horarios, políticas de acceso).
- **Módulos de analítica y visualización avanzada:** construir sobre la base de datos de visitas y eventos para desarrollar paneles de analítica que ofrezcan indicadores de ocupación, patrones horarios, detección de horas pico y recomendaciones de mejora operativa. La arquitectura basada en Kafka facilita que estos módulos se implementen como consumidores adicionales de eventos, sin alterar el microservicio de visión ni el backend principal.
- **Ampliación de funcionalidades en la aplicación móvil:** extender la aplicación dirigida al usuario final para incluir funciones como reserva de plazas, notificaciones personalizadas, integración con métodos de pago o visualización de recomendaciones de horarios y sitios con menor ocupación, siempre teniendo en cuenta las restricciones y necesidades del contexto institucional.
- **Seguridad y aspectos éticos:** profundizar en mecanismos de seguridad y privacidad, incluyendo la gestión de listas negras o blancas de vehículos, la detección de patrones anómalos de acceso, y el tratamiento adecuado de los datos personales asociados a las matrículas y a los usuarios. Esto es especialmente relevante si se contempla el uso del sistema en entornos fuera del ámbito estrictamente educativo.
- **Documentación y transferencia académica:** consolidar guías, manuales y material docente basados en ANPR-VISION, de manera que el sistema pueda ser reutilizado en futuras cohortes del SENA como plataforma de aprendizaje. Esto podría incluir prácticas guiadas sobre visión por computador, microservicios, mensajería distribuida, despliegue en la nube y pruebas de software.

El desarrollo de estas líneas de trabajo no solo permitiría mejorar técnicamente el prototipo, sino también consolidar a ANPR-VISION como un punto de partida para investigaciones más profundas y para experiencias formativas que acerquen a los aprendices a los desafíos reales de la ingeniería de software en sistemas distribuidos y de visión por computador.

APÉNDICE

APÉNDICES

En la Tabla ?? se presenta un resumen de las principales tecnologías utilizadas en ANPR-VISION y el rol que

cumplen dentro de la arquitectura. Los números de versión corresponden a las configuraciones empleadas durante el desarrollo del trabajo de grado y pueden variar en implementaciones posteriores.

Tabla II: Resumen de tecnologías y componentes empleados en ANPR-VISION.

| Componente                             | Tecnología                                                  | Rol en el sistema                                                                                                        |
|----------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Microservicio de visión                | Python + YOLOv5 + EasyOCR + ByteTrack                       | Procesamiento de video, detección de vehículos y matrículas, OCR y seguimiento de objetos.                               |
| Bus de mensajería                      | Apache Kafka                                                | Transporte asíncrono de eventos de detección entre el microservicio de visión y el backend de gestión.                   |
| Backend de gestión                     | .NET (API REST)                                             | Lógica de negocio para registrar visitas, gestionar parqueaderos, vehículos, tarifas y notificaciones.                   |
| Portal web                             | Angular                                                     | Interfaz para operadores y administradores: monitoreo de detecciones, consultas de visitas y configuración del sistema.  |
| Aplicación móvil                       | Tecnologías basadas en Angular/Ionic (según implementación) | Interfaz para usuarios finales: consulta de vehículos, historial de accesos y saldos asociados.                          |
| Comunicación en tiempo cercano al real | SignalR                                                     | Envío de notificaciones desde el backend hacia los clientes web y móviles cuando se produce un evento relevante.         |
| Base de datos (desarrollo / pruebas)   | Motor relacional en contenedor Docker                       | Almacenamiento de datos maestros (parqueaderos, cámaras, usuarios, vehículos) y transacciones (visitas, notificaciones). |
| Base de datos (producción)             | Amazon RDS (motor relacional)                               | Almacenamiento persistente y administrado de la información en el entorno productivo.                                    |
| Infraestructura de despliegue          | Docker, docker-compose, máquina virtual en AWS              | Contenerización de servicios, orquestación básica por entorno y despliegue en la nube.                                   |
| Integración y entrega continua         | Jenkins                                                     | Automatización de compilación, construcción de imágenes Docker y despliegue por entornos (develop, QA, staging, prod).   |

El Listado ?? muestra un ejemplo simplificado del mensaje JSON que el microservicio de visión envía al tópico de detecciones en Apache Kafka. Este formato se utiliza como contrato de integración entre la capa de visión y el backend de gestión.

Listing 1: Ejemplo de evento de detección publicado en Kafka.

```
{
  "eventId": "b3a9f1c0-4f8a-4f1b-9d93-8b0c1f5a1234",
  "plate": "ABC123",
  "confidence": 0.92,
  "cameraId": "CAM-ENTRADA-01",
  "parkingId": 1,
  "timestampUtc": "2025-05-12T15:43:21Z",
}
```

```
8  "trackId": 37,
9  "eventType": "DETECTION",
10 "imageMeta": {
11   "frameIndex": 1542,
12   "resolution": "1280x720"
13 }
14 }
```

En el backend, un componente consumidor de Kafka interpreta este mensaje, verifica la validez de la matrícula, determina si corresponde al inicio o a la actualización de una visita, y actualiza la base de datos en consecuencia.

A modo de referencia, en la Tabla ?? se resumen algunos de los endpoints más relevantes expuestos por la API de gestión de parqueaderos. La lista no es exhaustiva, pero ilustra el tipo de operaciones que consumen el portal web y la aplicación móvil.

Tabla III: Ejemplos de endpoints de la API de gestión.

| Método HTTP | Ruta                                      | Descripción                                                                                         |
|-------------|-------------------------------------------|-----------------------------------------------------------------------------------------------------|
| GET         | /api/visits/by-parking/{parkingId}        | Obtiene el listado de visitas (entradas y salidas) registradas para un parqueadero.                 |
| GET         | /api/visits/by-vehicle/{plate}            | Obtiene el historial de visitas asociadas a una matrícula específica.                               |
| GET         | /api/parking/{id}                         | Obtiene la información básica de un parqueadero (nombre, dirección, parámetros de configuración).   |
| GET         | /api/camera/by-parking/{parkingId}        | Obtiene las cámaras configuradas para un parqueadero, incluyendo sus URLs de conexión.              |
| GET         | /api/notifications/by-parking/{parkingId} | Obtiene las notificaciones pendientes o recientes relacionadas con un parqueadero.                  |
| PUT         | /api/notifications/{id}/read              | Marca una notificación como leída en nombre de un usuario o rol específico.                         |
| GET         | /api/users/me                             | Devuelve la información básica del usuario autenticado y sus roles.                                 |
| GET         | /api/reports/summary/{parkingId}          | Obtiene un resumen de visitas y ocupación en un intervalo de tiempo dado (según la implementación). |

Como se describe en la Sección ??, las pruebas del prototipo se realizaron utilizando un teléfono móvil Android con la aplicación *IP Webcam* como fuente de video, emulando el comportamiento de una cámara IP. A continuación se resumen los pasos básicos seguidos para la configuración, con el fin de que otros grupos puedan reproducir el montaje experimental:

1. Instalar la aplicación *IP Webcam* desde la tienda oficial de aplicaciones del dispositivo.
2. Conectar el teléfono y el servidor donde se ejecuta el microservicio de visión a la misma red local (por ejemplo, una red Wi-Fi institucional).

3. En *IP Webcam*, configurar la resolución de video y la orientación de la cámara según el escenario de prueba (entrada de parqueadero, distancia aproximada al vehículo, etc.).
4. Iniciar el servidor de video en la aplicación y anotar la URL generada (por ejemplo, `http://IP_DEL_TELEFONO:8080/video`).
5. Registrar dicha URL en el módulo de configuración de cámaras del sistema ANPR-VISION, de manera que el microservicio de visión consuma este flujo igual que consumiría el de una cámara IP dedicada.

Desde el punto de vista de la arquitectura, esta configuración no requiere cambios en el código del microservicio: la diferencia entre una cámara IP física y el teléfono con *IP Webcam* se limita a la dirección y a los parámetros de la fuente de video. Esto permite validar el sistema en contextos educativos sin necesidad de adquirir cámaras especializadas.

Para facilitar la reproducción del entorno de despliegue utilizado en el proyecto, se elaboraron archivos `docker-compose` específicos para los entornos de desarrollo, QA, *staging* y producción. Aunque los archivos completos se omiten aquí por extensión, su estructura general es la siguiente:

- Definición de servicios para el backend .NET, el portal web Angular y la base de datos relacional.
- Definición opcional de servicios para Apache Kafka, su coordinador (cuando aplica) y el microservicio de visión.
- Configuración de redes internas (por ejemplo, `anpr-net-develop`, `anpr-net-prod`) para aislar los servicios por entorno.
- Uso de ficheros `.env` por entorno para gestionar credenciales, cadenas de conexión y parámetros específicos (URLs de cámaras, tópicos de Kafka, claves de JWT, etc.).

La combinación de estos archivos con las canalizaciones de Jenkins descritas en el cuerpo del artículo permite reconstruir el entorno con un esfuerzo moderado, constituyendo un recurso útil para futuras cohortes que deseen utilizar ANPR-VISION como base para nuevas actividades académicas o investigaciones aplicadas.